

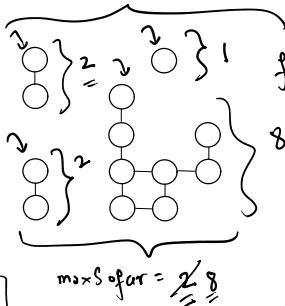
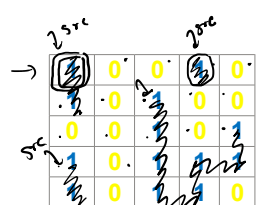
Max Area of Island

Given a two-dimensional grid that contains 0's and 1's where 0 represents water and 1 represents land. We define an island as a group of 1's (representing land) connected four-directionally (horizontal & vertical). Design an algorithm to find the area of the largest island - area of an island is the number of 1's it contains.

Example

Input :

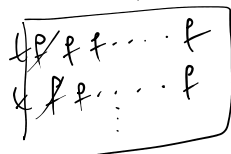
Output : 8



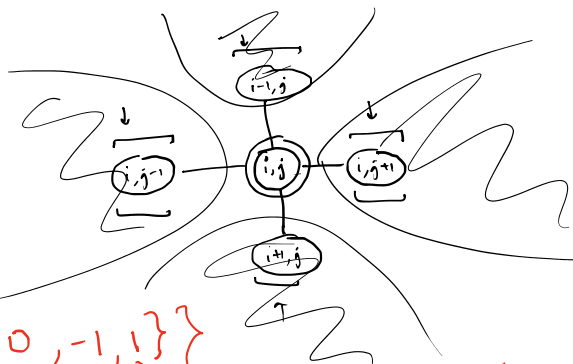
finding the
max area
of island

finding the size
of the largest
component

maxSoFar = 8

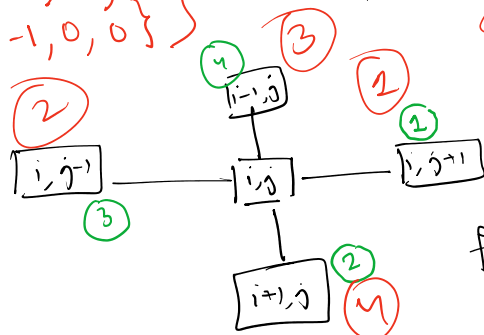


m x n



$$dx = \{0, 0, -1, 1\}$$

$$dy = \{1, -1, 0, 0\}$$



$$i + 0 \Rightarrow i$$

$$j + 1 \Rightarrow j + 1$$

$$dx = \{0, 1, 0, -1\}$$

$$dy = \{1, 0, -1, 0\}$$

for (k=0; k<4; k++) {
 $\Rightarrow nx = i + dx[k];$
 $\Rightarrow ny = j + dy[k];$
 $dfs(nx, ny, ...)$
}

$$nx = i + 0 \Rightarrow i$$

$$ny = j + -1 \Rightarrow j - 1$$

$$k = 0 \Rightarrow 3$$

$$nx = i + -1 \Rightarrow i - 1$$

$$ny = j + 0 \Rightarrow j$$

$$nx = i + 1 \Rightarrow i + 1$$

$$ny = j + 0 \Rightarrow j$$

[HW] Making a Largest Island

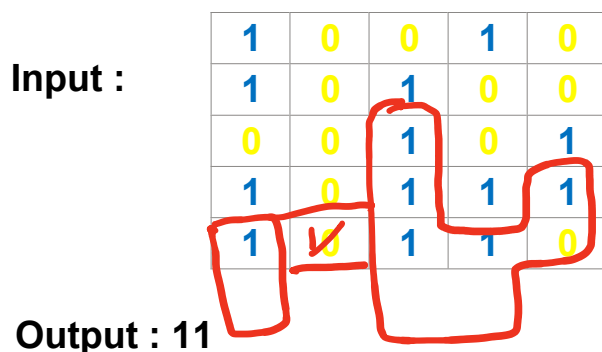
[editorial] <https://leetcode.com/problems/making-a-large-island/editorial/>



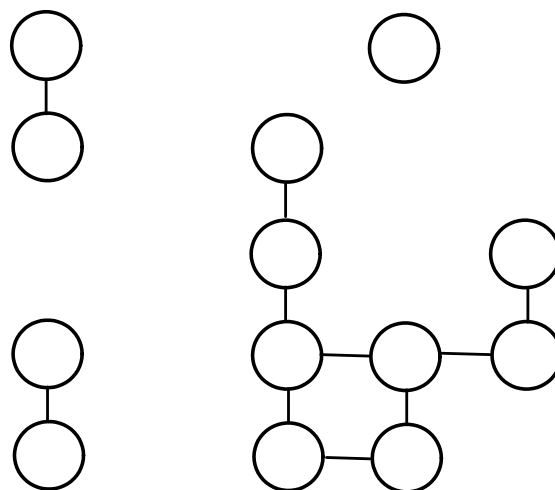
Making a Largest Island

Given a two-dimensional grid that contains **0's** and **1's** where **0** represents **water** and **1** represents **land**. We define an **island** as a **group of 1's** (representing land) connected **four-directionally** (horizontal & vertical). Design an algorithm to find the **area of the largest island** - area of an island is the **number of 1's** it contains such that you can change at-most one of the **0's** into a **1**.

Example

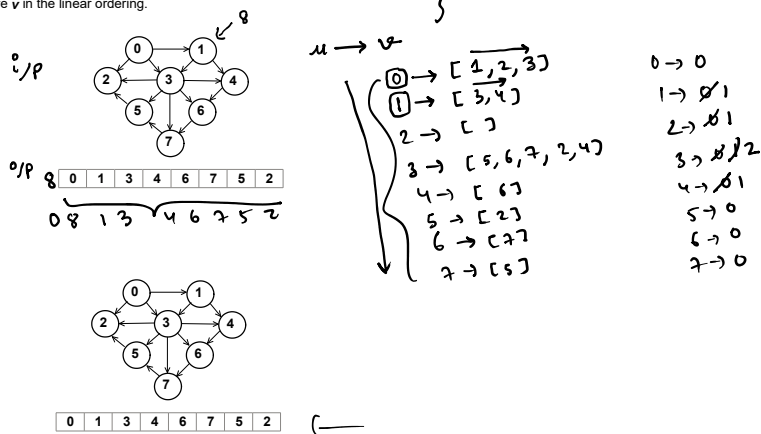


1	0	0	1	0
1	0	1	0	0
0	0	1	0	1
1	0	1	1	1
1	0	1	1	0



Topological Sorting of a DAG

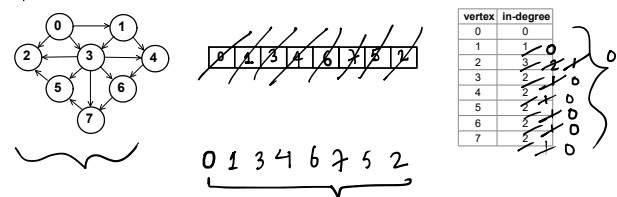
Topological Sorting of a **directed acyclic graph** or DAG is a linear ordering of its vertices such that for each directed edge from vertex u to vertex v in the graph, u comes before v in the linear ordering.



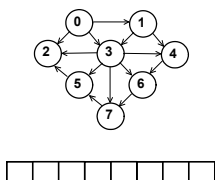
BFS Implementation (Kahn's Algorithm)

To perform topological sorting of a DAG using the BFS algorithm, we will maintain

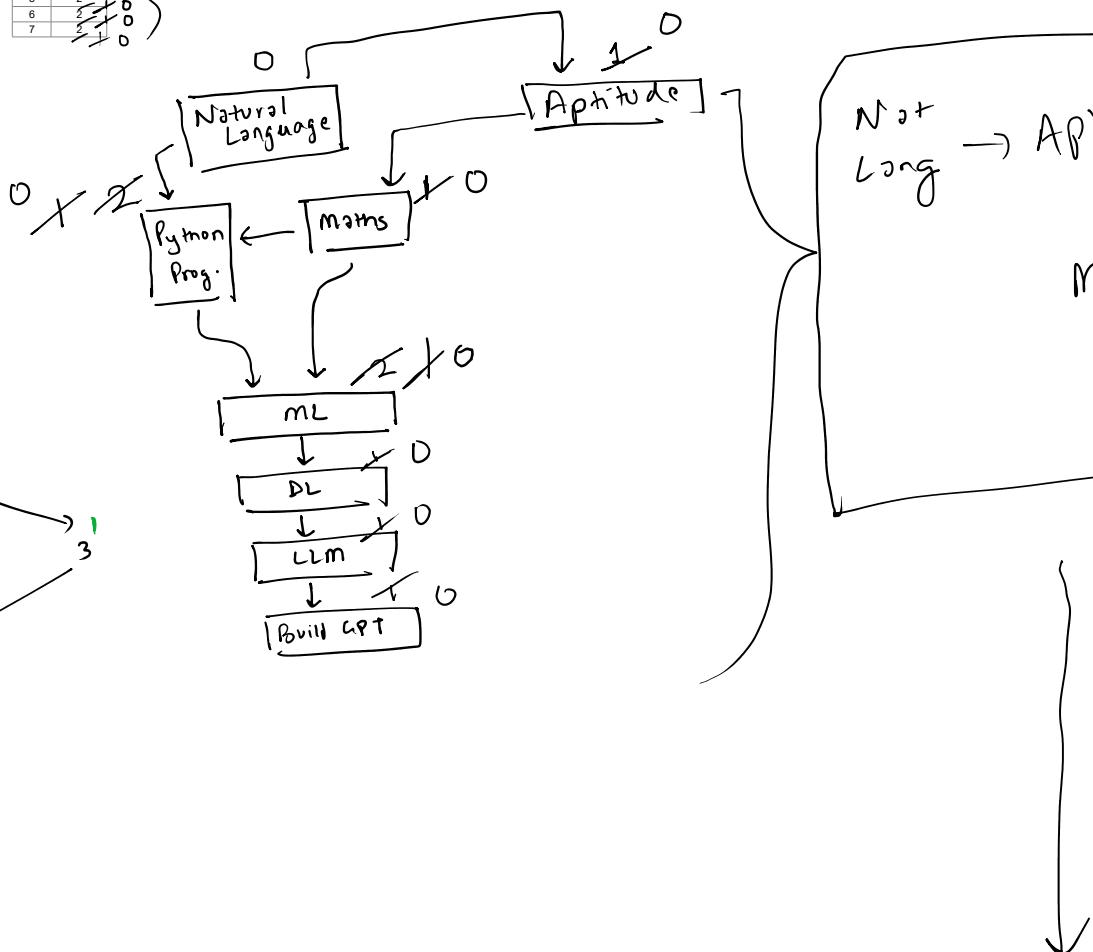
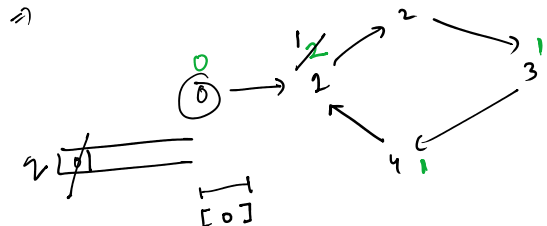
- a **Queue** to track nodes whose dependencies have been resolved i.e. in-degree is 0.
- a **HashMap** to store a mapping b/w vertices and their in-degrees.

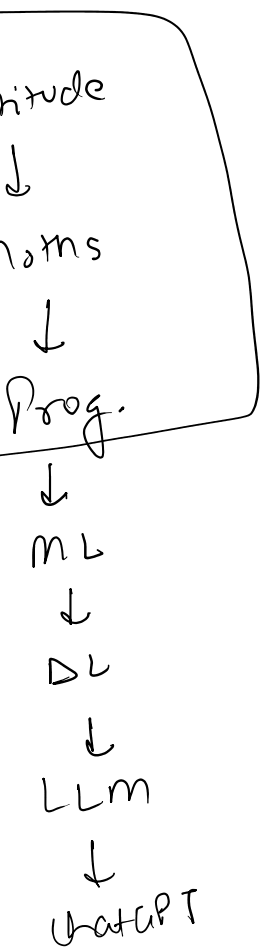


[HW] DFS Implementation



Cycle Detection using Topological Sorting





Course Schedule II

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example

numCourses

4

prerequisites

10

20

31

32

0, 1, 2, 3

0 → 1 → 3
0 → 2 → 3

numCourses

5

prerequisites

10

20

31

32

Alien Dictionary

a b c d e f . . . z

There is a new alien language that uses the English alphabet. However, the order among the letters is unknown to you.

You are given a list of strings `words` from the alien language's dictionary, where the strings in `words` are sorted lexicographically by the rules of this new language.

Return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there is no solution, return `""`. If there are multiple solutions, return any of them.

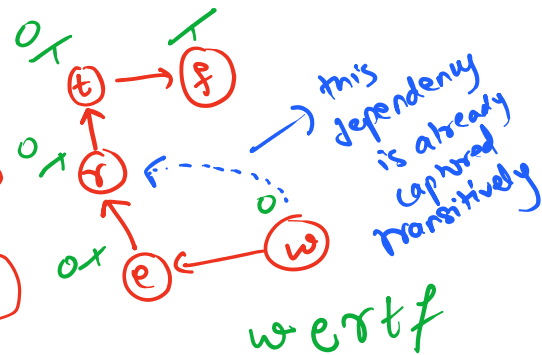
Example 1:

Input: words = ["wrt","wrf","er","ett","rftt"]

Output: "wertf"

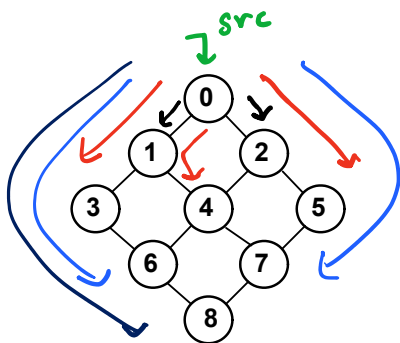
sorted by alien lang. rules
 $wrt < wrf \Rightarrow w \rightarrow r$

$wrt < wrf \Rightarrow t < f : t \rightarrow f$
 $wrf < er \Rightarrow w < e : w \rightarrow e$
 $er < ett \Rightarrow r < t : r \rightarrow t$
 $ett < rftt \Rightarrow e < r : e \rightarrow r$



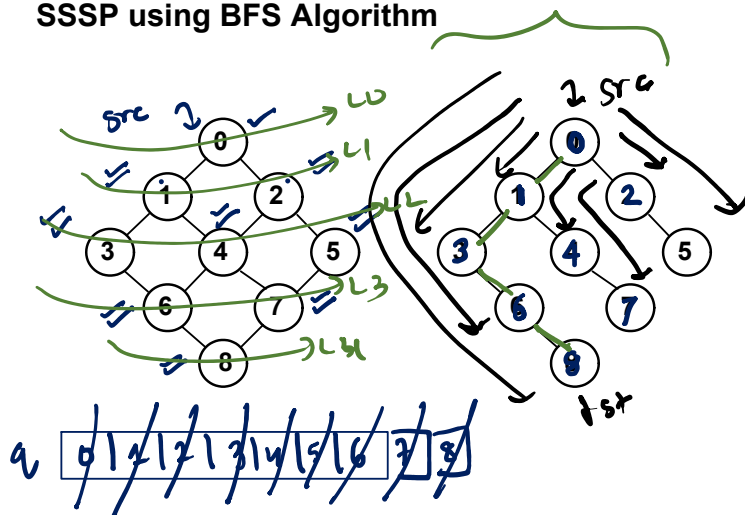
Single Source Shortest Path (SSSP)

Given a **connected** graph and **source** vertex design an algorithm to find the length of the **shortest path** from the **source vertex** to all the **other** vertices.



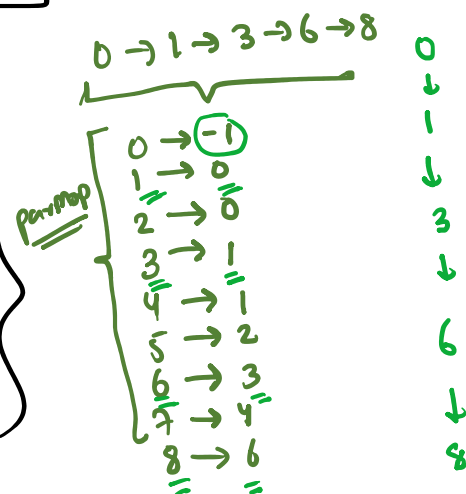
vertex	distance
0	0
1	1
2	1
3	2
4	2
5	2
6	3
7	3
8	4

SSSP using BFS Algorithm

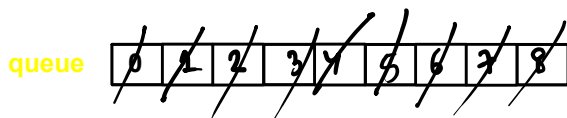
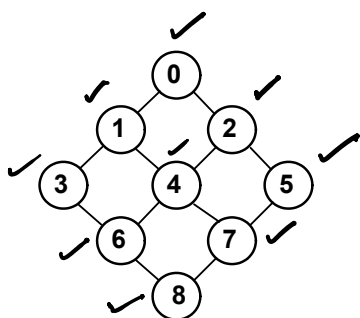


src = 0 dst = 8
4

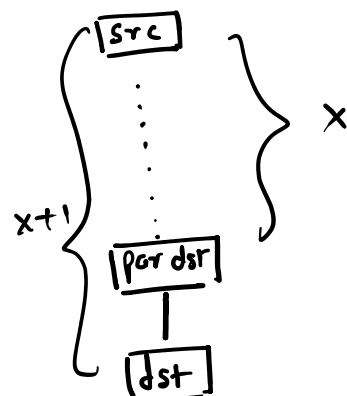
vertex	distance
0	0
1	1
2	1
3	2
4	2
5	2
6	3
7	3
8	4



BFS Implementation for SSSP



vertex	distance
0	0
1	1
2	1
3	2
4	2
5	2
6	3
7	3
8	4



Snakes and Ladders Game

Given a Snakes & Ladders board of size **N** that contains multiple snakes and ladders. You are also given their **starting and ending positions**. Design an algorithm to find the **minimum no. of throws** of a six-sided die required to reach the **Nth** cell of the board from its **1st** cell such that as per game rules -

- If you are bitten by a snake (by reaching at a cell having a **snake head**) you will **go down** to the cell at which the **tail of the snake** ends.
- If you reach a cell at which a **ladder starts**, you will **climb** the ladder.

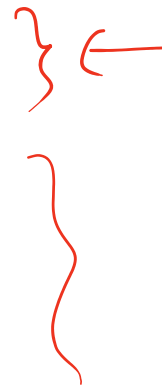
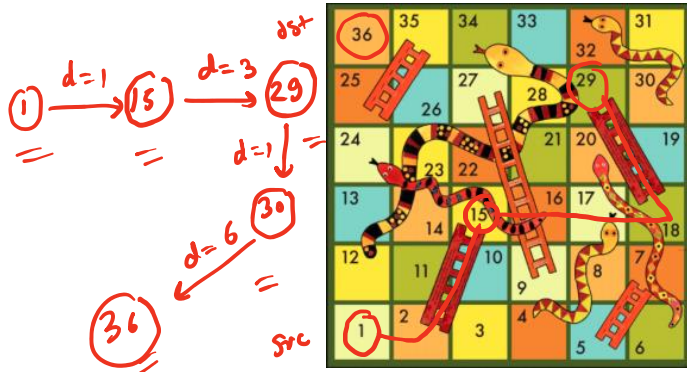
Example

Input : **N = 36**

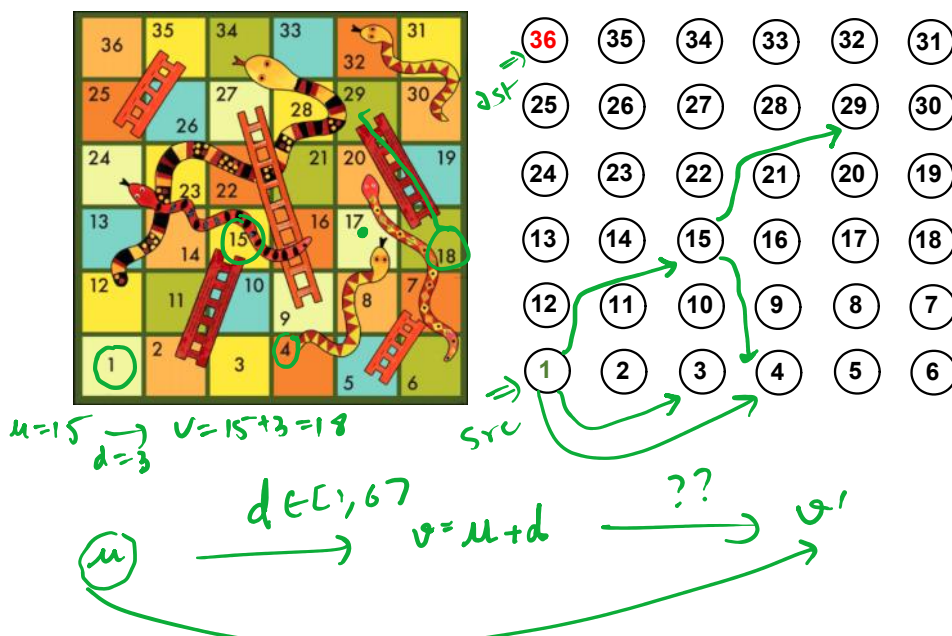
ladderPos = [(2, 15), (5, 7), (9, 27), (18, 29), (25, 35)]

snakePos = [(17, 4), (20, 6), (24, 16), (32, 30), (34, 12)]

Output : 4



$$u = 15 \xrightarrow{d=2} v = 15 + 2 = 17$$





All Nodes Distance K in Binary Tree

Given the **root** of a binary tree, a **target** node and an integer **K**, return an array of values of all the **nodes at distance K** from the **target** node.

